
FlangParallelAnalyzer

A Simple Guide to the Project

Automatic Loop Parallelism Analysis for Fortran

Aryan Gupta
Arushi Vaidya
Arpita

Department of Computer Science and Engineering
RV College of Engineering, Bengaluru

Contents

1	What is this project about?	2
2	The Three Possible Outcomes	2
3	How Does It Work? (The Pipeline)	3
3.1	Phase 1 — What does this loop look like?	3
3.2	Phase 2 — What memory does this loop touch?	4
3.3	Phase 3 — Are array subscripts independent?	4
3.4	Phase 4 — Is this a reduction?	4
3.5	Phase 5 — Final conservative decision	5
4	A Full Worked Example	5
5	What the Tool is Made Of	6
6	Test Results	6
7	The HTML Report	7
8	Known Limitations	7
9	What's Next	7
10	Summary	8

1. What is this project about?

Imagine you have a Fortran program that processes a huge array of numbers — say, a weather simulation running over a million grid points. Inside the program, there is a loop that looks like this:

```
do i = 1, 1000000
    temperature(i) = temperature(i) * 1.05
end do
```

Your computer today has 8, 16, or even 64 cores sitting idle. Instead of running this loop one step at a time, you could split it across all cores and finish it $64\times$ faster. That is what **parallel computing** is — doing many things at the same time.

In Fortran, you tell the compiler to parallelize a loop by adding a single line:

```
!$OMP PARALLEL DO          <- this one line splits the work
do i = 1, 1000000
    temperature(i) = temperature(i) * 1.05
end do
```

The problem: Not every loop can be parallelized safely. If one iteration of the loop depends on a result computed by a previous iteration, running them at the same time gives the **wrong answer** — silently, with no error message.

Figuring out which loops are safe to parallelize, by hand, for a codebase with hundreds of loops, takes hours and is error-prone.

FlangParallelAnalyzer (FPA) solves this automatically. You give it a Fortran file, and it tells you — for every loop — whether it is safe to parallelize, needs a special clause, or is genuinely unsafe.

2. The Three Possible Outcomes

For every loop it finds, FPA gives one of three verdicts:

SAFE — Go ahead and parallelize.

Each iteration of the loop works on completely independent data. No two iterations ever read or write the same memory cell.

Example:

```
do i = 1, n
    b(i) = a(i) * 2.0      ! iteration 3 touches b(3) and a(3)
end do                   ! iteration 7 touches b(7) and a(7)
                        ! they never overlap -> SAFE
```

Hint emitted: !\$OMP PARALLEL DO

REDUCTION — Safe with a special clause.

The loop adds up (or multiplies) into a single accumulator variable. OpenMP can handle this pattern natively.

Example:

```
do i = 1, n
    total = total + a(i) * b(i)    ! "total" accumulates across
                                !     iters
end do
```

Hint emitted: !\$OMP PARALLEL DO REDUCTION(+:total)

UNSAFE — Do not parallelize.

One iteration reads a value that was written by another iteration. Parallelizing this would produce wrong results.

Example:

```
do i = 2, n
    a(i) = a(i) + a(i-1)    ! iteration 5 reads a(4), which
end do                    ! iteration 4 is still computing
    -> UNSAFE
```

Hint emitted: ! Cannot parallelize

3. How Does It Work? (The Pipeline)

FPA does not guess. It reads the compiled form of the Fortran program (called **FIR** — Fortran Intermediate Representation) and runs a five-step analysis on every loop it finds.

Why FIR and not the Fortran source directly?

Fortran source code has many tricky syntax rules (implicit variable types, fixed vs free format, etc.). FIR is a clean, structured version produced by the compiler where every array access, every variable, and every loop is spelled out explicitly. It is much easier to analyze.

3.1. Phase 1 — What does this loop look like?

FPA first collects basic facts about the loop:

- What are the loop bounds? (e.g., 1 to n)
- How deep is the nesting? (is this inside another loop?)
- How many operations are in the body?

At this point, the verdict is still *unknown*.

3.2. Phase 2 — What memory does this loop touch?

FPA walks every read (*load*) and write (*store*) inside the loop and builds a list of every variable it sees:

Variable	Read?	Written?	External?
a (array)	Yes	No	Yes (passed in)
b (array)	No	Yes	Yes (passed in)
i (counter)	Yes	Yes	No (local)

External means the variable came from outside the loop — i.e., it is a function argument or a variable declared before the loop. These are the ones that matter for dependency analysis.

3.3. Phase 3 — Are array subscripts independent?

This is the core of the analysis. FPA looks at how each array is indexed and asks: “**Does iteration *i* touch the same element as iteration *j*?**”

It traces every subscript back through the compiler IR to find out whether it is:

- **Exactly the loop variable *i*:** each iteration touches a unique element — independent, no conflict.

```
a(i)      <- subscript = i  -> safe, each iter unique
```

- **The loop variable plus or minus a constant ($i \pm k$):** iteration *i* reads the element that iteration *i-k* will write — that is a **loop-carried dependency** and the loop is immediately marked UNSAFE.

```
a(i-1)    <- subscript = i-1  -> iter i reads what iter i-1
      writes
```

- **Something else** (e.g., an indirect index like `a(idx(i))`): FPA cannot prove it is safe, so it passes to Phase 4.

3.4. Phase 4 — Is this a reduction?

If Phase 3 left the verdict unknown (usually because a scalar variable is both read and written), Phase 4 checks for the **reduction pattern**:

```
! The pattern FPA looks for:
old_value = load(accumulator)           ! step 1: read
new_value = old_value + something        ! step 2: arithmetic
store(new_value, accumulator)           ! step 3: write back
```

If this exact chain is found, the loop is marked REDUCTION and the correct REDUCTION(+:var) or REDUCTION(*:var) clause is emitted.

3.5. Phase 5 — Final conservative decision

Any loop still undecided after Phase 4 gets a fallback verdict:

- If no external variable is written at all: the loop is **SAFE** (a read-only loop cannot have dependencies).
- Otherwise: **UNSAFE** — FPA does not know enough to guarantee correctness, so it plays it safe.

Key design principle: FPA would rather call a safe loop “unsafe” (you miss a speedup) than call an unsafe loop “safe” (you get wrong answers). **Zero false positives** is guaranteed.

4. A Full Worked Example

Here is what FPA produces when run on a real Fortran file:

Input:

```
subroutine scale(a, b, n)
  real, intent(in)  :: a(n)
  real, intent(out) :: b(n)
  integer :: i
  do i = 1, n
    b(i) = a(i) * 2.0
  end do
end subroutine
```

Command:

```
flang-new -fc1 -emit-fir scale.f90 -o /tmp/scale.fir
./build/tools/fpa-tool/fpa-tool /tmp/scale.fir
```

Output:

```
[FlangParallelAnalyzer] Function: _QPscale
-----
Loop #1 @ scale.fir:6
Bounds : [1 .. ? step 1]
Access : ext-reads=1  ext-writes=1  ext-readwrites=0
         [R]  array  -- a
         [W]  array  -- b
Status  : SAFE
Hint    : !$OMP PARALLEL DO
Reason  : Independent per-element access: each iteration reads
         a(i) and writes b(i) with no overlap across iterations
         .
-----
```

The developer can now confidently add `!$OMP PARALLEL DO` above the loop and get correct parallel execution.

5. What the Tool is Made Of

File	Size	What it does
LoopParallelAnalysis.cpp	471 lines	The main analysis engine. Runs all 5 phases on every loop.
AccessClassifier.cpp	165 lines	Walks through every memory read/write in a loop and records it.
main.cpp (fpa-tool)	71 lines	The command-line tool. Loads the file and runs the analysis.
report.py	1259 lines	Python script that runs the tool on all test files and generates the interactive HTML report.
run_tests.py	250 lines	Automated test runner — checks that all 35 test cases pass.

Technologies used:

- **LLVM 18** — the industry-standard compiler toolkit that FPA is built on top of.
- **MLIR** — the part of LLVM that lets us write our own analysis pass and walk through code systematically.
- **Flang** — the LLVM Fortran compiler that converts .f90 source to FIR (the format FPA reads).
- **C++ 17** — the language the analysis logic is written in.
- **Python 3** — used for the test runner and HTML report generator.

6. Test Results

FPA was tested on a suite of 35 handcrafted Fortran programs covering every major loop pattern. Every single test gets the correct verdict.

Category	Test cases	Correct
Embarrassingly parallel (SAFE)	13	13
Scalar reductions (REDUCTION)	5	5
Loop-carried dependencies (UNSAFE)	10	10
Complex / indirect patterns (UNSAFE conservatively)	7	7
Total	35	35 (100%)

Zero false positives. FPA never tells you a loop is safe when it is not. The worst it can do is miss a parallelization opportunity — it never gives wrong answers.

7. The HTML Report

Running `python3 scripts/report.py` generates a browser-based report with three panels:

1. **File Browser** — lists all Fortran files with a colour-coded badge (**SAFE** / **REDUCTION** / **UNSAFE**) for each loop.
2. **Source View** — shows the Fortran source with an inline chip on the `do` line indicating the verdict and a confidence score (e.g. HIGH 90%).
3. **Analysis Detail** — shows exactly which of the 5 phases fired, what memory accesses were found, and the full reason for the verdict.

This lets a developer immediately see what was decided and *why*, without reading any compiler IR.

8. Known Limitations

FPA is a research prototype. A few patterns are not yet handled:

- **In-place same-index updates:** `a(i) = a(i) * 3` is actually safe (each iteration touches its own element) but FPA conservatively marks it UNSAFE because it cannot confirm the read happens before the write without extra analysis.
- **Multi-dimensional arrays:** `c(i,j)` subscripts are not yet traced, so nested loops over 2D arrays get UNSAFE.
- **Function calls:** if the loop body calls an external function, FPA does not know if that function has side effects, so it cannot guarantee safety.
- **Min/Max reductions:** only `+` and `*` reductions are detected. `MIN/MAX` accumulations are not yet supported.

9. What's Next

- Fix the in-place update false negative with a read-before-write check on the value chain.
- Add support for `MIN/MAX` reductions.
- Trace multi-dimensional subscripts axis by axis.
- Analyze function side-effects interprocedurally.
- Automatically insert `!$OMP` directives into the Fortran source (currently the tool only suggests them).

10. Summary

In one paragraph:

FlangParallelAnalyzer is a static analysis tool that reads compiled Fortran programs and automatically determines, for every DO loop, whether it is safe to run in parallel. It works by analyzing the Fortran Intermediate Representation (FIR) produced by the LLVM Flang compiler, running a five-phase pipeline that checks array subscript patterns, detects scalar reductions, and applies a conservative fallback. It produces a verdict (SAFE, REDUCTION, or UNSAFE) along with an OpenMP directive hint and a full explanation. On a 35-case test suite it achieves 100% accuracy with zero false positives — it never tells you a loop is safe when it is not.

Aryan Gupta Arushi Vaidya Arpita
Department of Computer Science and Engineering
RV College of Engineering, Bengaluru

<https://github.com/AryanGupta21/FlangParallelAnalyzer>